



# DSP Overview of the uberClock Project

**Authors:**

Ahmed Imamovic
Tarik Hamedovic

**Mentored by:**
Dan Gisselquist

# Contents

# 1 Introduction

This document covers the DSP concepts and tools used in uberClock project. It covers the architecture and algorithms used to excite and control our XTAL oscillator.

# 2 Hardware architecture

The complete FPGA hardware architecture is shown in Figure 1. The design can be divided into two clock domains: a high-speed domain operating at 65,MHz and a low-speed domain operating at 10,kHz. The low-speed domain interfaces with the embedded CPU through the LiteX CSR bus, allowing firmware to monitor and control the signal-processing chain.

The signal path begins in the excitation generation stage, where the digital excitation waveform is created. In the transmit chain, the signal is upsampled and digitally upconverted to the desired carrier frequency. The resulting waveform is sent to a 14-bit DAC, whose output drives the analog front-end containing the crystal (XTAL) under test.

The response of the crystal is captured by the analog front-end and digitized using a 12-bit ADC. The sampled signal then enters the receive chain, where it is digitally downconverted and downsampled to baseband. The resulting low-bandwidth signals are exposed through CSR registers, enabling the CPU firmware to perform measurements.

There are 5 channels consisting of the TX and RX paths, which are used to single out 5 different modes of the crystal. In our case, those are

- C100 - 3.333 MHz
- B100 - 3.660 MHz
- A100 - 6.224 MHz
- C300 - 10.000 MHz
- B300 - 10.981 MHz

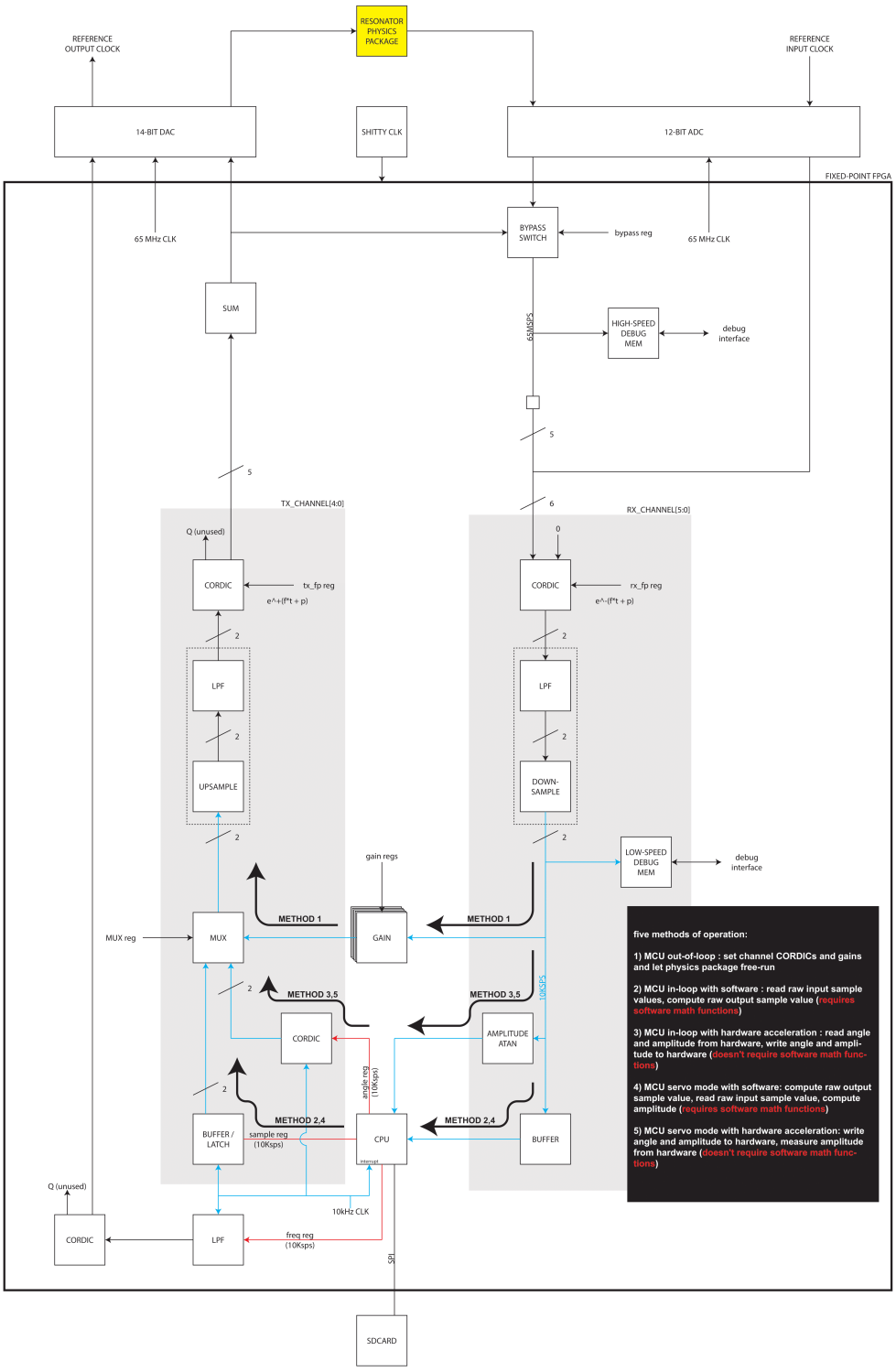


Figure 1: UberClock architecture designed by ZipCPU

## 2.1 RX channel

The RX channel in UberClock is a per-mode digital receive path that takes the common ADC input and translates one selected spectral component to low requency/baseband for further processing. Functionally, the RX channel consists of three main parts:

- A CORDIC-based quadrature downconverter. The input sample is rotated by the generated phase, producing in-phase and quadrature components. This is the digital mixing stage that shifts the selected input tone from RF/IF down to near baseband.
- A decimation and low-pass filtering chain. Both I and Q branches pass through downsampling filter. That filter chain contains:
  - a CIC decimator,
  - a CIC compensation filter,
  - a half-band low-pass filter.

Its purpose is to remove mixer images and out-of-band content, while reducing the sample rate so later processing is cheaper and cleaner.

### 2.1.1 CORDIC Downconverter

A key component of the UberClock receive chain is the CORDIC-based digital downconverter. The purpose of the downconverter is to translate the digitized intermediate-frequency (IF) signal produced by the ADC to baseband, where the signal can be processed at a significantly lower bandwidth. This operation generates the in-phase ($I$) and quadrature ($Q$) signal components that contain the amplitude and phase information required for crystal characterization.

The Coordinate Rotation Digital Computer (CORDIC) algorithm is particularly well suited for FPGA implementations because it performs trigonometric operations using only additions, subtractions, and bit-shift operations, eliminating the need for dedicated multipliers. When configured in Rotation Mode (RM), a CORDIC can be used to implement a quadrature mixer while requiring fewer hardware resources than conventional DDS- and multiplier-based architectures [1].

Figure 2 shows the generic RM CORDIC architecture, while Figure 3 illustrates its application as a digital downconverter. As described by Valls et al. [1], the received IF signal $r(n)$ is connected to the $Y_0$ input of the CORDIC, while the $X_0$ input is set to zero. The rotation angle $\theta$ is generated by a numerically controlled oscillator operating at the desired local oscillator frequency $f_c$. By rotating the vector $(0, r(n))$, the CORDIC performs the same operation as a conventional quadrature mixer.

The resulting in-phase and quadrature components are obtained from the $Y_N$ and $X_N$ outputs, respectively, and are given by

$$I(n) = K\, r(n) \cos(f_c \pi n), \tag{1}$$

$$Q(n) = -K\, r(n) \sin(f_c \pi n), \tag{2}$$

where $K$ is the constant scaling factor introduced by the CORDIC rotation process [1]. These equations show that the input signal is mixed with orthogonal cosine and sine carriers, producing the baseband $I$ and $Q$ components required for further processing.

Compared to a conventional implementation based on a direct digital synthesizer (DDS), lookup tables, and multiple multipliers, a CORDIC-based downconverter significantly reduces FPGA resource utilization while maintaining equivalent functionality. Following downconversion, the generated $I$ and $Q$ signals are passed through the decimation stages of the receive chain before being exposed to the embedded CPU through LiteX Control and Status Registers (CSRs) for measurement and analysis.

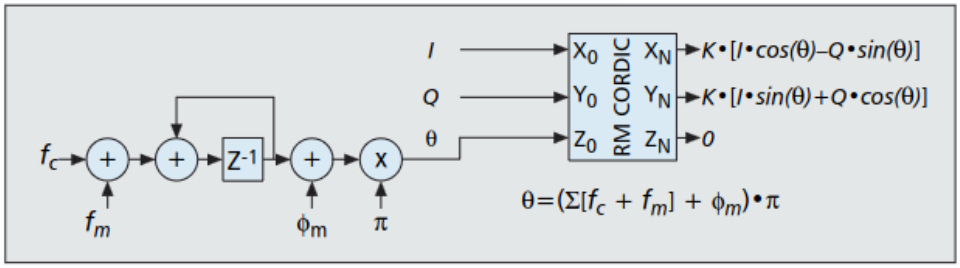


Figure 2: Generic Rotation Mode (RM) CORDIC architecture[1].

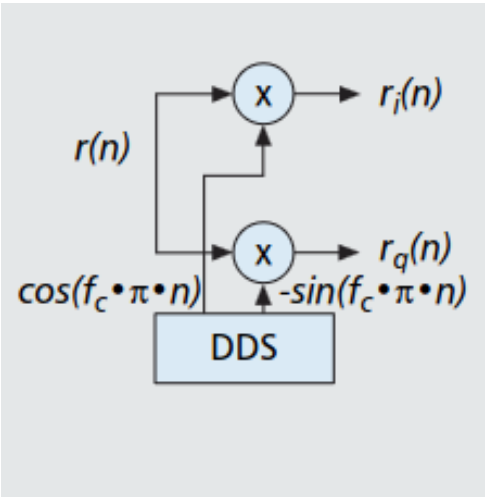


Figure 3: CORDIC-based quadrature downconverter[1].

### 2.1.2 Downsampling filter

Downsampling filter is used not only to reduce the sample rate, but to do that without corrupting the signal. After mixing to baseband, the signal still contains unwanted high-frequency components, including mixer images and broadband noise. If the rate were reduced immediately, those components would alias back into the band of interest. The downsampling filter prevents that.

The sample rate is from 65MHz to 10kHz in three stages. The table 1 shows the structure, as well as the type of the filer, decimation rate (R), number of taps and register widths for input, output and coefficient words.

Table 1: Downsampling filter chain

| DOWN | TYPE | R | IN_W | OUT_W | TAPS | STAGES | REG_W | CF_W |
|---|---|---|---|---|---|---|---|---|
| 1 | CIC | 1625 | 12 | 12 | – | 4 | – | – |
| 2 | COMP | 2 | 12 | 16 | 34 | – | – | 15 |
| 3 | HB | 2 | 16 | 16 | 95 | – | – | 19 |

The frequency response of the resulting filter (zoomed-in baseband) and the quantization effects can be seen on Figure 4.

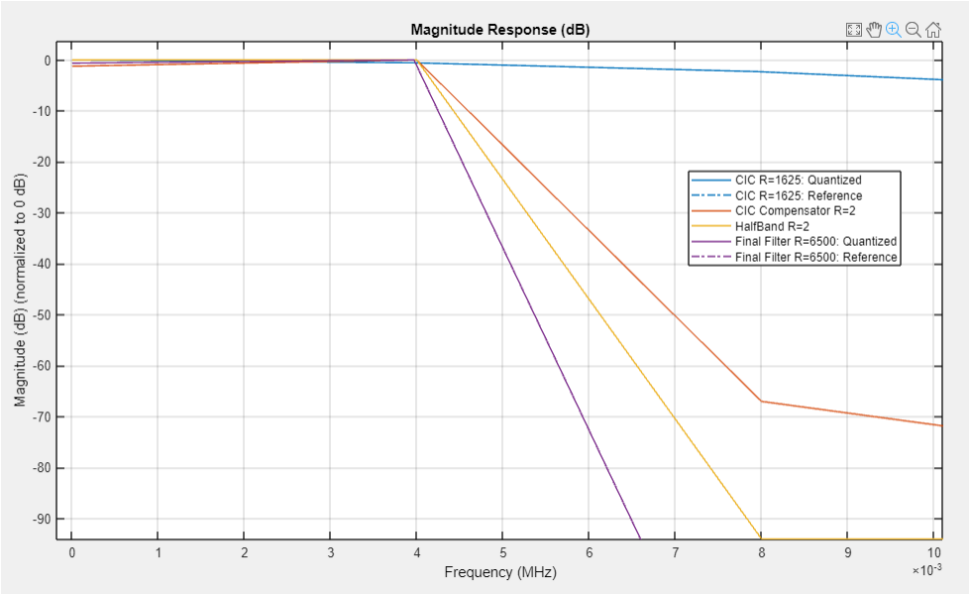


Figure 4: Frequency response of the downsampling filter.

## 2.2 TX path

The TX path performs the inverse operation of the RX path. It takes a low-rate complex baseband signal and shifts it back up to the carrier region so it can contribute to the final drive waveform.

Conceptually, the TX path has three main functions:

- **Baseband input processing.** The low-rate $I$ and $Q$ components coming from the receive/control side represent the desired amplitude and phase of the selected resonant mode.
- **Upsampling and interpolation.** Because the baseband signal exists at a reduced sample rate, it must be brought back to the high system sample rate. This is accomplished through interpolation filtering, which inserts new samples and suppresses the spectral images created by the upsampling process.
- **Quadrature upconversion.** A digital local oscillator provides a phase reference, and the complex baseband vector is rotated back to the desired carrier frequency. This operation converts the low-rate baseband signal into a bandpass waveform centered around the target mode frequency.

### 2.2.1 Upsampling filter

The upsampling filter is the stage that prepares a low-rate baseband signal for transmission at the high system sample rate. After the receive and control stages, the signal exists as a slowly varying baseband I/Q waveform. That is the correct information, but it is available at a reduced sample rate. Before upconversion, the signal must be brought back to the high-rate domain.

In UberClock, the TX interpolation filter is a multistage chain composed of a half-band interpolator, a CIC-compensation interpolator, and a final CIC interpolator. The first stage begins waveform reconstruction at low cost, the second stage corrects the passband shaping needed for the CIC response, and the last stage provides the large sample-rate increase efficiently. Table 2 summarizes our upsampling filter.

Table 2: Upsampling filter chain

| UP | TYPE | R | IN_W | OUT_W | TAPS | STAGES | REG_W | COEFF_W |
|---|---|---|---|---|---|---|---|---|
| 1 | HB | 2 | 16 | 16 | 119 | – | – | 19 |
| 2 | COMP | 2 | 16 | 16 | – | – | – | 16 |
| 3 | CIC | 1625 | 16 | 14 | – | 5 | 71 | – |

The frequency response of the resulting filter (zoomed-in baseband) and the quantization effects can be seen on Figure 5.

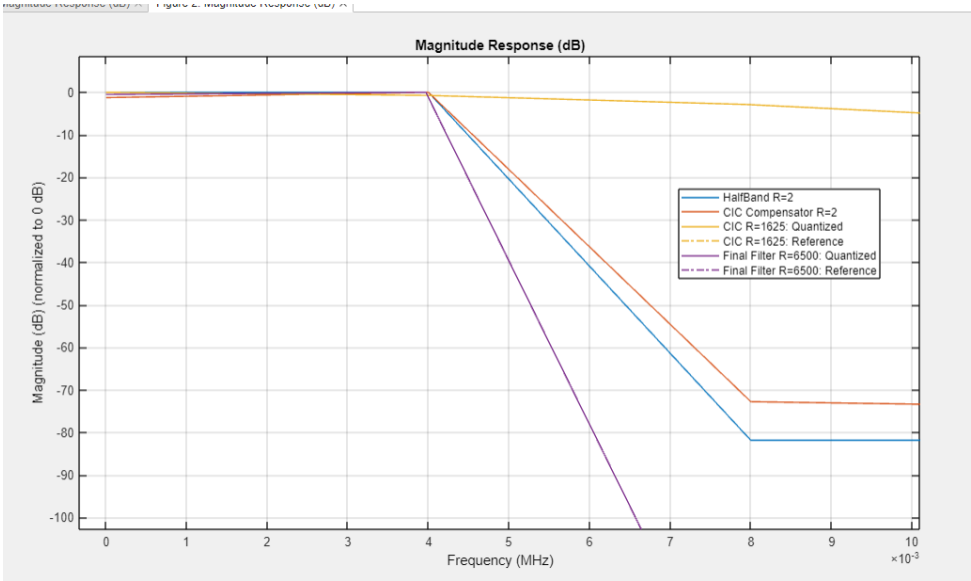


Figure 5: Frequency response of the upsampling filter.

### 2.2.2 CORDIC Upconverter

After the receive-side processing and control algorithms have determined the desired excitation signal, the transmit chain must reconstruct a high-rate waveform suitable for driving the DAC. This is accomplished using a CORDIC-based quadrature upconverter.

As described by Valls et al. [1], a CORDIC operating in Rotation Mode (RM) can be used to perform digital upconversion without requiring dedicated multipliers or sine/cosine lookup tables. Instead, the operation is implemented using only additions, subtractions, and bit-shift operations, making it highly efficient for FPGA implementations.

Figure 2 shows the generic RM CORDIC architecture. In the digital upconverter configuration, the baseband in-phase and quadrature signals are connected to the $X_0$ and $Y_0$ inputs of the CORDIC, respectively. The rotation angle $\theta$ is generated by a numerically controlled oscillator operating at the desired carrier frequency $f_c$.

The CORDIC performs a rotation of the input vector according to

$$X_N = K\left[I\cos(\theta) - Q\sin(\theta)\right], \tag{3}$$

$$Y_N = K\left[I\sin(\theta) + Q\cos(\theta)\right], \tag{4}$$

where $I$ and $Q$ are the baseband signal components and $K$ is the constant CORDIC scaling factor [1].

For the specific case of digital upconversion, the baseband signals $s_i(n)$ and $s_q(n)$ are applied to the $X_0$ and $Y_0$ inputs, while the phase input is driven by the carrier frequency $f_c$. The resulting bandpass signal is obtained from the $X_N$ output and can be expressed as

$$s(n) = K\left[s_i(n)\cos(f_c\pi n) - s_q(n)\sin(f_c\pi n)\right]. \tag{5}$$

This operation translates the complex baseband signal to the desired carrier frequency while preserving its amplitude and phase information. Compared to a conventional implementation based on a DDS, lookup tables, and multiple multipliers, the CORDIC-based solution significantly reduces FPGA resource utilization while providing equivalent functionality.

In the UberClock transmit chain, the interpolated baseband signal is upconverted using this CORDIC structure before being sent to the DAC. The resulting waveform excites the selected crystal mode at its target frequency, completing the closed-loop transmit/receive architecture.

## 2.3 Debug Capture and Data Export

The hardware includes two complementary capture paths. The low-speed path is used for quick interactive inspection through CSRs, while the high-speed path records full-rate samples into DDR3 memory and exports them to the host for offline analysis.

### 2.3.1 High-Speed Capture with DDR3

High-speed capture is used when the signal must be observed at the full system sample rate, approximately 65 MHz. Instead of reading samples directly through the CPU, the design writes a capture stream into external DDR3 memory using DMA. This makes it possible to record long windows of ADC or DSP data for FFT analysis, waveform plotting, and validation of the receive chain.

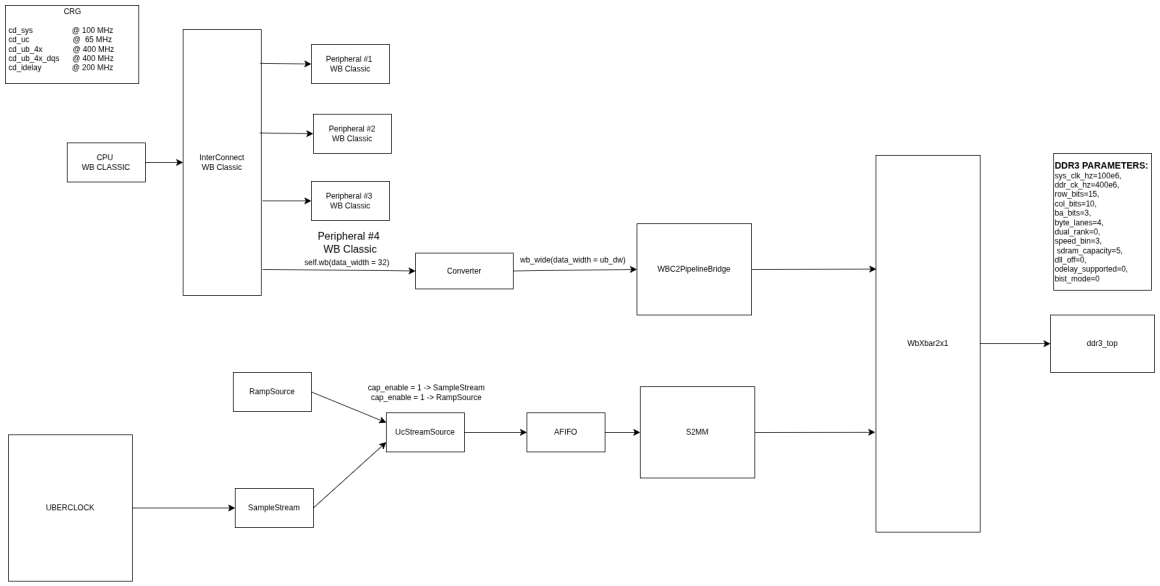


Figure 6: High-speed capture architecture using an asynchronous FIFO, DMA, Wishbone interconnect, and DDR3 memory.

The high-speed capture path crosses from the high-rate UC domain into the system domain before writing to memory:

```
DSP / ADC (UC domain)
        |
  capture stream
        |
  async FIFO (UC to SYS)
        |
  DMA (zipdma_s2mm)
        |
  Wishbone crossbar
        |
       DDR3
```

The asynchronous FIFO is the clock-domain boundary. The UC domain produces the high-rate samples, while the system domain owns the DMA engine, memory interconnect, and DDR3 controller.

A typical capture setup from the firmware console is:

```
dma_addr0 0xA0000000
dma_addr1 0x00000000
dma_inc 1
dma_size 0

cap_enable 1
cap_beats 4194304

dma_req 1
```

Here, `cap_beats` counts DMA beats rather than individual ADC samples. The DDR3 address must be aligned, and DDR3 must be initialized before starting the capture. Ramp mode should be used first to validate the memory path before capturing real data.

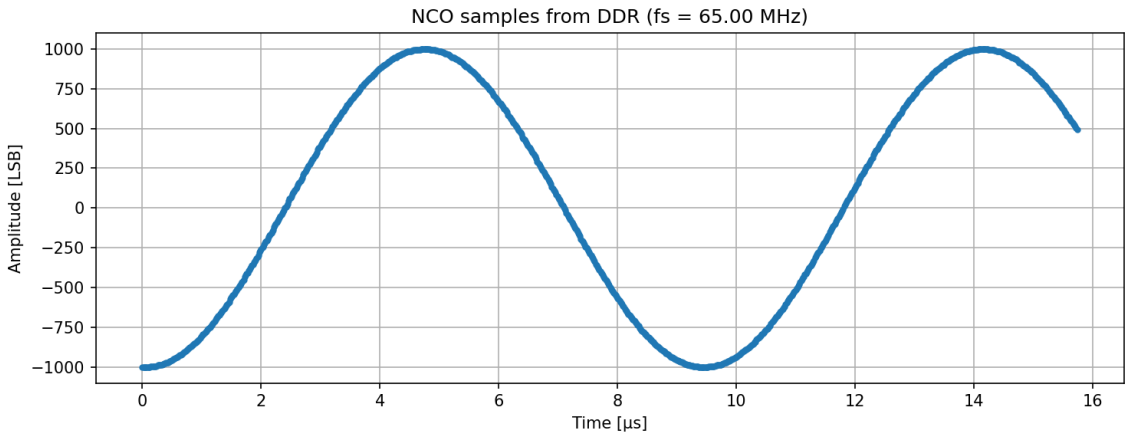


Figure 7: Example full-rate high-speed capture reconstructed on the host.

### 2.3.2 UDP Export for High-Speed Capture

After the high-speed capture is stored in DDR3, the data is transferred to the host over UDP. This step makes the DDR3 capture usable outside the FPGA by reconstructing the captured memory image as a binary file on the host.

```
DDR3 memory
     |
DMA reader
     |
 UDP stream
     |
host receiver
     |
capture.bin
     |
analysis / plotting
```

The main receiver script is:

```
python 3.build/scripts/receive_udp_capture.py
```

The receiver listens for UDP packets, checks packet headers, reconstructs the binary stream, and writes `capture.bin`. The captured file can then be converted or plotted using the helper scripts:

- `capture_bin_to_txt.py` converts binary samples into a readable numeric format.
- `plot_capture_bin.py` plots the raw waveform.
- `plot_capture_lanes.py` inspects multi-lane or interleaved capture data.
- `plot_csv_fft.py` performs frequency-domain analysis.
- `plot_csv_fft_coherent.py` performs coherent FFT analysis for more precise measurements.

A normal host-side workflow is:

```
python 3.build/scripts/receive_udp_capture.py
python 3.build/scripts/capture_bin_to_txt.py capture.bin
python 3.build/scripts/plot_capture_bin.py capture.bin
python 3.build/scripts/plot_csv_fft.py capture.csv
```

Before trusting a high-speed capture, the output should be checked for the expected file size, packet loss, ramp continuity, and waveform continuity.

### 2.3.3 Low-Speed CSR Capture

The low-speed capture path is intended for quick firmware-driven debugging. It uses a small on-chip capture buffer that is accessed through LiteX CSRs, so it does not require DDR3, DMA, or UDP transfer.

```
DSP / debug source
       |
 capture buffer
       |
      CSRs
       |
  CPU / UART
```

The firmware selects a capture source, arms the capture, waits for completion, and then reads the samples back through CSR registers. A typical interactive workflow is:

```
cap_arm
cap_status
cap_dump
```

This path is useful for validating small datasets, checking polarity and scaling, debugging DSP stages, and observing short transient behavior. Its main limitations are the small buffer depth, slow CPU readout, and inability to capture high-rate continuous signals.

In practice, low-speed capture is used first when debugging logic interactively. High-speed DDR3 capture is used once longer records, real signal analysis, or full-rate samples are needed.

# 3 Software algorithms

## 3.1 Tracking Algorithm

The tracker is a small software loop wrapped around the FPGA DSP path. The FPGA does the expensive, timing-sensitive work: CORDIC mixing, filtering, downsampling, and FIFO movement. The firmware only sees a low-rate stream. It measures where the tones landed, then writes a new `phase_down_<channel>` value through the LiteX CSRs.

That split is intentional. The signal path stays deterministic in RTL, while the search rule can still be changed from C without resynthesizing the FPGA.

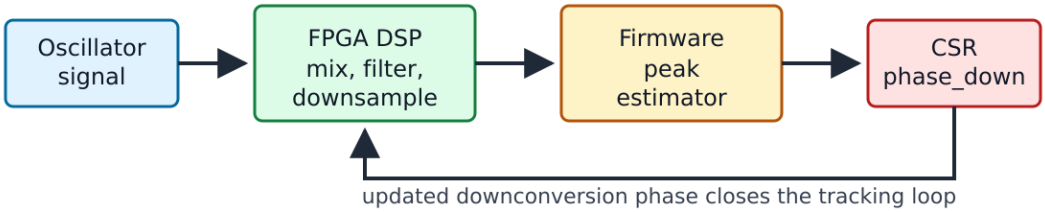


Figure 8: Tracking loop from the oscillator signal through the FPGA DSP path, firmware estimator, and CSR phase update.

The current firmware has two tracking commands:

**`track3`** A coarse search. It sweeps one downconversion frequency until the expected three-tone shape is visible in the downsampled spectrum.

**`trackq_start` / `trackq_stop`** A fine tracker. Once the coarse point is close, it periodically measures the same three tones, estimates the peak offset, and nudges channels 1, 2, and 3.

### 3.1.1 Signal model

During tracking the firmware injects, or expects to see, three tones around a known baseband center:

$$f_L = f_c - h, \qquad f_C = f_c, \qquad f_R = f_c + h$$

Here `f_c` is usually 1 kHz and `h` is the per-channel spacing. The default spacing is 10 Hz for channel 1 and 30 Hz for channels 2 and 3 in the `trackq` path.

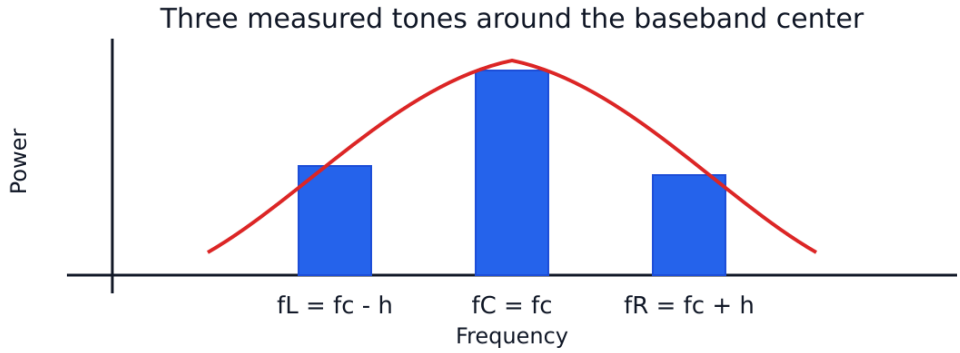


Figure 9: Three-tone power measurement around the baseband center frequency.

The downconverter setting is a 26-bit phase increment. Firmware converts between Hertz and the CSR value with:

$$\text{phase\_inc} = \left\lfloor \frac{f_{\text{down}}\, 2^{26}}{65\,000\,000} \right\rfloor$$

and the inverse conversion used for prints is:

$$f_{\text{down}} \approx \frac{\text{phase\_inc}\, 65\,000\,000}{2^{26}}$$

So one LSB is just under 1 Hz.

### 3.1.2 Power measurement

`track3` runs a KISS FFT over a captured block and reads the bins nearest `f_L`, `f_C`, and `f_R`. For a capture length `N` and downsampled rate `F_s`:

$$k(f) = \text{round}\left(\frac{fN}{F_s}\right)$$

The power in a complex FFT bin is:

$$P[k] = \Re\{X[k]\}^2 + \Im\{X[k]\}^2$$

For `trackq` the firmware uses a narrow correlator-style measurement on the captured samples instead of relying on a single FFT bin.

$$I(f) = \sum_{n=0}^{N-1} x[n] \cos\left(2\pi \frac{fn}{F_s}\right)$$

$$Q(f) = -\sum_{n=0}^{N-1} x[n] \sin\left(2\pi \frac{fn}{F_s}\right)$$

$$P(f) = I(f)^2 + Q(f)^2$$

The code also measures one bin-width on either side and folds that into the reported band power:

$$P_b(f) = P(f) + \frac{P(f - \Delta f_{\text{bin}}) + P(f + \Delta f_{\text{bin}})}{2}$$

where:

$$\Delta f_{\text{bin}} = \frac{F_s}{N}$$

### 3.1.3 Coarse lock rule

The coarse search is deliberately conservative. A candidate point is accepted only when the center tone is strongest and both side tones are present:

$$P_C > P_L, \qquad P_C > P_R$$

The side tones must also sit inside the acceptance window used by `track3_triplet_match`:

$$0.02 P_C \le \min(P_L, P_R)$$

$$\max(P_L, P_R) \le 0.95 P_C$$

$$\min(P_L, P_R) \ge 0.40 \max(P_L, P_R)$$

### 3.1.4 `track3` coarse sweep

`track3` starts from a requested downconversion frequency and walks forward in fixed Hertz steps:

```
track3 <channel> <start_hz> [step_hz] [max_steps] [N] [center_hz] [delta_hz]
```

Typical setup:

```
fft_fs 10000
track3 1 10002950 10 400 2048 1000 20
```

For each trial point it:

1. converts the candidate frequency to a `phase_down` increment,
2. writes the CSR and commits the update,
3. throws away a short settling window,
4. captures `N` downsampled samples from the selected channel,
5. measures `P_L`, `P_C`, and `P_R`,
6. stops if the three-bin rule passes.

If no lock is found, the firmware restores the original phase increment.

### 3.1.5 Fine tracking with a three-point parabola

Once the tones are close, `trackq` treats the three power values as samples of a peak. With equally spaced samples at `f_c-h`, `f_c`, and `f_c+h`, the vertex estimate is:

$$\hat{f} = f_c + h\frac{P_L - P_R}{2(P_L - 2P_C + P_R)}$$

The denominator should be negative for a real peak. If it is not, the firmware does not trust the parabola.

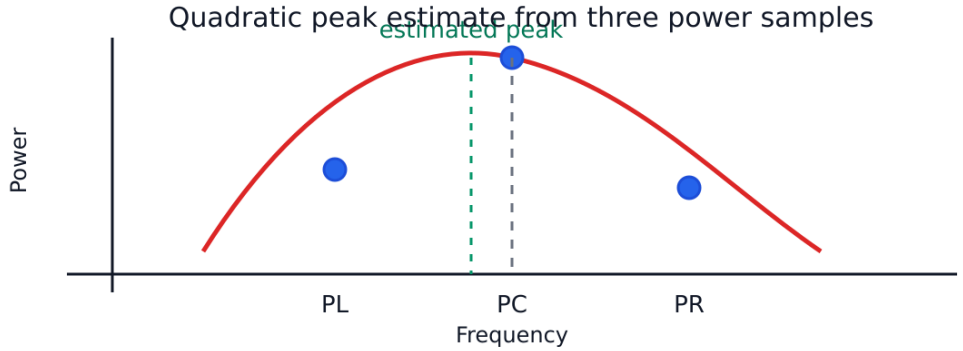


Figure 10: Quadratic interpolation of the resonance peak from three power samples.

The measured baseband error is:

$$e = \hat{f} - f_c$$

The controller filters the error:

$$e_f[n] = e_f[n-1] + \frac{1}{4}\left(e[n] - e_f[n-1]\right)$$

then applies a small proportional gain:

$$u[n] = \frac{1}{4} e_f[n]$$

The fractional part is kept in an accumulator, and the integer-Hertz correction is clamped before it is written back:

$$\Delta f_{\text{down}} = \text{clamp}\left(\lfloor u_{\text{acc}} \rfloor, -2\,\text{Hz}, +2\,\text{Hz}\right)$$

### 3.1.6 Weak measurements

Sometimes the three samples do not pass the full confidence test. The firmware still tries to make a cautious correction if the side powers are clearly unbalanced:

$$e_{\text{weak}} \approx \frac{P_R - P_L}{P_R + P_L} \frac{h}{4}$$

There is a deadband around zero side imbalance, and the weak-mode correction is clamped to only 1 Hz.

### 3.1.7 Runtime cadence

`trackq` is serviced from `uberclock_poll`. The update interval is `TRACKQ_INTERVAL_TICKS`, currently 20000 downsample ticks. With the usual `fft_fs 10000` setup, that is a two-second tracking update.

Each update captures the three tracked channels together:

```
trackq_start <f1> <f2> <f3> [N] [center_hz] [delta_ch1_hz] [delta_ch2_hz] [
    delta_ch3_hz]
trackq_probe [N] [center_hz] [delta_hz]
trackq_stop
```

`trackq_probe` is the useful bench command when tuning. It runs one capture and prints the three powers and the estimated vertex without waiting for the periodic background log.

### 3.1.8 Example application

The following example demonstrates how the tracking algorithm is used to lock onto and continuously follow three resonant oscillator modes. Each mode can drift over time because of temperature changes, aging, or other environmental effects.

The workflow is divided into two stages:

1. `track3` performs a coarse search and locates the approximate resonance.
2. `trackq_start` begins continuous fine tracking using quadratic interpolation around the detected peak.

**3 Tone Tracking** The tracking system positions the CPU-generated tones and CORDIC mixer frequencies near the resonant frequency so that the fine tracker can maintain lock on the moving peak.

**C300 Mode** From lab measurements, this tone is expected in the following frequency range:

```
10.003840 MHz to 10.004000 MHz
```

The serial console interface is:

```
track3 <ch> <start_hz> [step_hz] [max_steps] [N] [center_hz] [delta_hz]
```

First coarse iteration:

```
uberClock> track3 1 10002840 20 10 2048 1000 20
track3: ch=1 start=10002840 Hz step=20 Hz max_steps=10 N=2048 center=1000 Hz
    delta=20 Hz Fs=10000 Hz sig3={980,1000,1020} Hz
track3 step=0 phase_down_1=10002840 Hz inc=10327372 bins={201,205,209} pwr
    ={3535,554,323}
track3 step=1 phase_down_1=10002860 Hz inc=10327393 bins={201,205,209} pwr
    ={188,114,66}
track3 step=2 phase_down_1=10002880 Hz inc=10327414 bins={201,205,209} pwr
    ={51,33,42}
track3 step=3 phase_down_1=10002900 Hz inc=10327434 bins={201,205,209} pwr
    ={18,16,25}
track3 step=4 phase_down_1=10002920 Hz inc=10327455 bins={201,205,209} pwr
    ={15,25,25}
track3 step=5 phase_down_1=10002940 Hz inc=10327476 bins={201,205,209} pwr
    ={20,16,13}
track3 step=6 phase_down_1=10002960 Hz inc=10327496 bins={201,205,209} pwr
    ={16,32,17}
track3 lock: phase_down_1=10002960 Hz inc=10327496 center=1000 left=980 right
    =1020
```

Second refinement iteration:

```
uberClock> track3 1 10002940 1 20 2048 1000 10
track3: ch=1 start=10002940 Hz step=1 Hz max_steps=20 N=2048 center=1000 Hz delta
    =10 Hz Fs=10000 Hz sig3={990,1000,1010} Hz
track3 step=0 phase_down_1=10002940 Hz inc=10327476 bins={203,205,207} pwr
    ={60067,136499,18232}
track3 step=1 phase_down_1=10002941 Hz inc=10327477 bins={203,205,207} pwr
    ={75089,110155,16630}
track3 step=2 phase_down_1=10002942 Hz inc=10327478 bins={203,205,207} pwr
    ={67740,57706,12101}
track3 step=3 phase_down_1=10002943 Hz inc=10327479 bins={203,205,207} pwr
    ={74213,33618,10973}
track3 step=4 phase_down_1=10002944 Hz inc=10327480 bins={203,205,207} pwr
    ={94839,25360,12059}
track3 step=5 phase_down_1=10002945 Hz inc=10327481 bins={203,205,207} pwr
    ={129631,16819,4922}
track3 step=6 phase_down_1=10002946 Hz inc=10327482 bins={203,205,207} pwr
    ={4352,25171,68193}
track3 step=7 phase_down_1=10002947 Hz inc=10327483 bins={203,205,207} pwr
    ={6651,34312,72923}
track3 step=8 phase_down_1=10002948 Hz inc=10327484 bins={203,205,207} pwr
    ={9542,69438,91629}
```

```
track3 step=9 phase_down_1=10002949 Hz inc=10327485 bins={203,205,207} pwr
    ={8819,71914,90698}
track3 step=10 phase_down_1=10002950 Hz inc=10327486 bins={203,205,207} pwr
    ={8676,69903,84677}
track3 step=11 phase_down_1=10002951 Hz inc=10327487 bins={203,205,207} pwr
    ={10158,62017,76351}
track3 step=12 phase_down_1=10002952 Hz inc=10327488 bins={203,205,207} pwr
    ={15947,54087,63095}
track3 step=13 phase_down_1=10002953 Hz inc=10327489 bins={203,205,207} pwr
    ={24303,54736,49692}
track3 lock: phase_down_1=10002953 Hz inc=10327489 center=1000 left=990 right
    =1010
```

**A100 Mode** Expected frequency range:

```
6.261641 MHz to 6.271641 MHz
```

Example coarse lock:

```
uberClock> track3 2 6261000 1000 10 2048 1000 100
track3: ch=2 start=6261000 Hz step=1000 Hz max_steps=10 N=2048 center=1000 Hz
    delta=100 Hz Fs=10000 Hz sig3={900,1000,1100} Hz
track3 step=0 phase_down_2=6261000 Hz inc=6464132 bins={184,205,225} pwr
    ={0,108,0}
track3 step=1 phase_down_2=6262000 Hz inc=6465164 bins={184,205,225} pwr={0,74,0}
track3 step=2 phase_down_2=6263000 Hz inc=6466197 bins={184,205,225} pwr
    ={0,106,0}
track3 step=3 phase_down_2=6264000 Hz inc=6467229 bins={184,205,225} pwr={0,93,0}
track3 step=4 phase_down_2=6265000 Hz inc=6468262 bins={184,205,225} pwr={0,99,0}
track3 step=5 phase_down_2=6266000 Hz inc=6469294 bins={184,205,225} pwr
    ={0,105,0}
track3 step=6 phase_down_2=6267000 Hz inc=6470326 bins={184,205,225} pwr={0,95,0}
track3 step=7 phase_down_2=6268000 Hz inc=6471359 bins={184,205,225} pwr
    ={77,95,97}
track3 step=8 phase_down_2=6269000 Hz inc=6472391 bins={184,205,225} pwr
    ={96,69,121}
track3 step=9 phase_down_2=6270000 Hz inc=6473424 bins={184,205,225} pwr
    ={105,130,67}
track3 lock: phase_down_2=6270000 Hz inc=6473424 center=1000 left=900 right=1100
```

**C100 Mode** Expected frequency range:

```
3.388438 MHz to 3.388594 MHz
```

Example lock:

```
uberClock> track3 3 3387400 10 10 2048 1000 20
track3: ch=3 start=3387400 Hz step=10 Hz max_steps=10 N=2048 center=1000 Hz delta
    =20 Hz Fs=10000 Hz sig3={980,1000,1020} Hz
track3 step=0 phase_down_3=3387400 Hz inc=3497301 bins={201,205,209} pwr
    ={14621,15243,11202}
track3 step=1 phase_down_3=3387410 Hz inc=3497311 bins={201,205,209} pwr
    ={13495,13321,9311}
track3 step=2 phase_down_3=3387420 Hz inc=3497321 bins={201,205,209} pwr
    ={14570,10784,7988}
track3 step=3 phase_down_3=3387430 Hz inc=3497331 bins={201,205,209} pwr
    ={12620,9838,7987}
```

```
track3 step=4 phase_down_3=3387440 Hz inc=3497342 bins={201,205,209} pwr
    ={10305,8390,8350}
track3 step=5 phase_down_3=3387450 Hz inc=3497352 bins={201,205,209} pwr
    ={10714,9075,7482}
track3 step=6 phase_down_3=3387460 Hz inc=3497362 bins={201,205,209} pwr
    ={8867,9387,6490}
track3 lock: phase_down_3=3387460 Hz inc=3497362 center=1000 left=980 right=1020
```

**Continuous Tracking** Once the initial coarse locks are found, continuous tracking is started with:

```
trackq_start <f1> <f2> <f3> [N] [center_hz] [delta_ch1_hz] [delta_ch2_hz] [
    delta_ch3_hz]
```

Example:

```
uberClock> trackq_start 10002953 6270000 3387460 2048 1000 10 100 20
trackq_start: ch1=10002953 Hz ch2=6270000 Hz ch3=3387459 Hz N=2048 center=1000 Hz
     delta={10,100,20} Hz sig3={{990,1000,1010},{900,1000,1100},{980,1000,1020}}
    Hz interval=2 s
trackq hf vertex: ch1=10003952.591Hz ch2=6270999.743Hz ch3=3388454.227Hz
trackq hf vertex: ch1=10003952.591Hz ch2=6270989.468Hz ch3=3388449.384Hz
trackq hf vertex: ch1=10003952.591Hz ch2=6270977.848Hz ch3=3388445.509Hz
trackq hf vertex: ch1=10003951.622Hz ch2=6270966.016Hz ch3=3388444.541Hz
trackq hf vertex: ch1=10003950.059Hz ch2=6270960.719Hz ch3=3388443.572Hz
trackq hf vertex: ch1=10003949.441Hz ch2=6270955.188Hz ch3=3388442.604Hz
trackq hf vertex: ch1=10003949.540Hz ch2=6270953.199Hz ch3=3388441.635Hz
trackq hf vertex: ch1=10003949.685Hz ch2=6270951.865Hz ch3=3388441.635Hz
trackq hf vertex: ch1=10003949.685Hz ch2=6270945.502Hz ch3=3388437.761Hz
trackq hf vertex: ch1=10003949.685Hz ch2=6270945.502Hz ch3=3388436.792Hz
trackq hf vertex: ch1=10003949.685Hz ch2=6270868.985Hz ch3=3388412.578Hz
```

Using the low-speed debug signal capture, the three tracking tones can be observed directly in the FPGA debug channels.

The tracker attempts to keep the central 1 kHz tone aligned with the interpolated resonance peak.

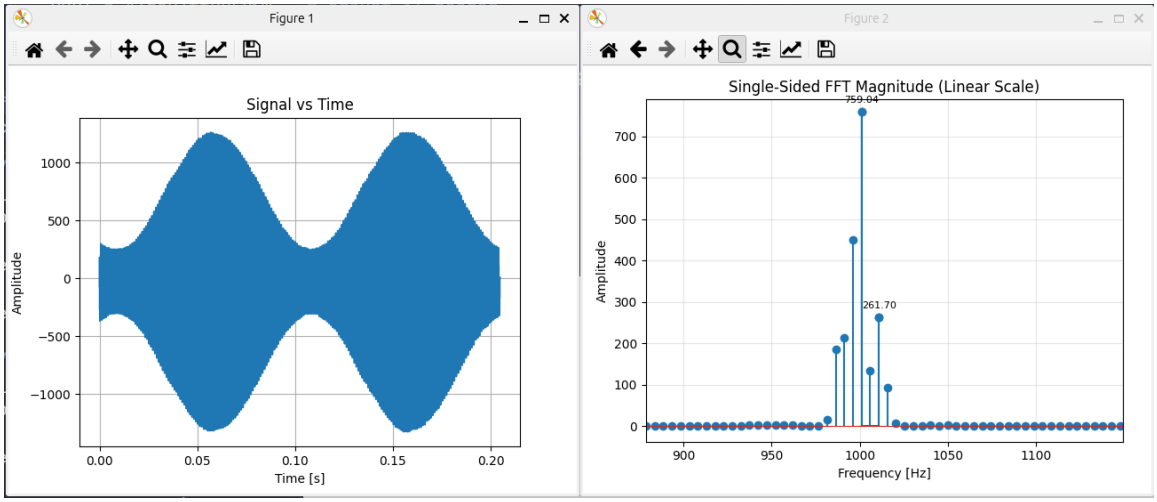


Figure 11: Low-speed debug capture for tracking channel 1.

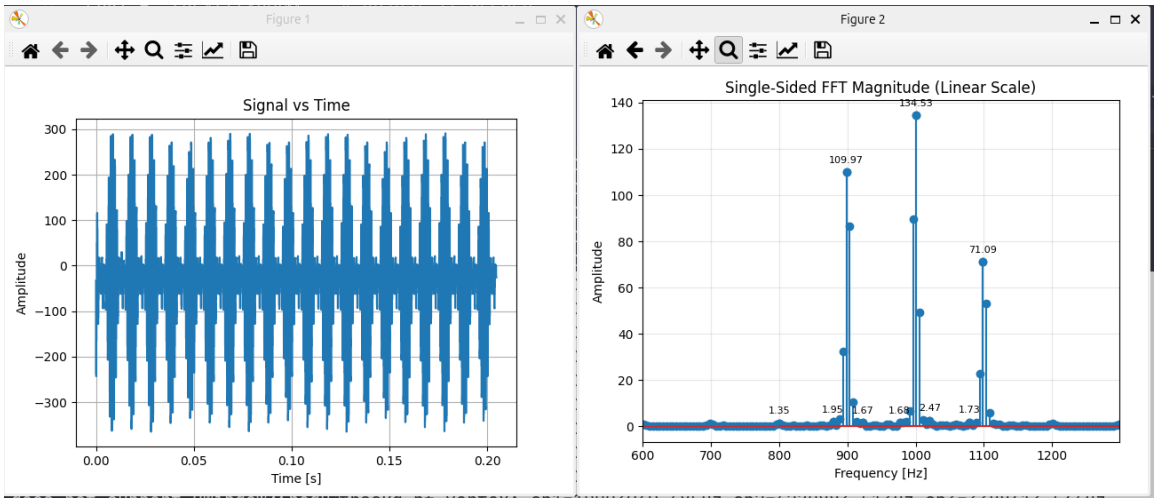


Figure 12: Low-speed debug capture for tracking channel 2.

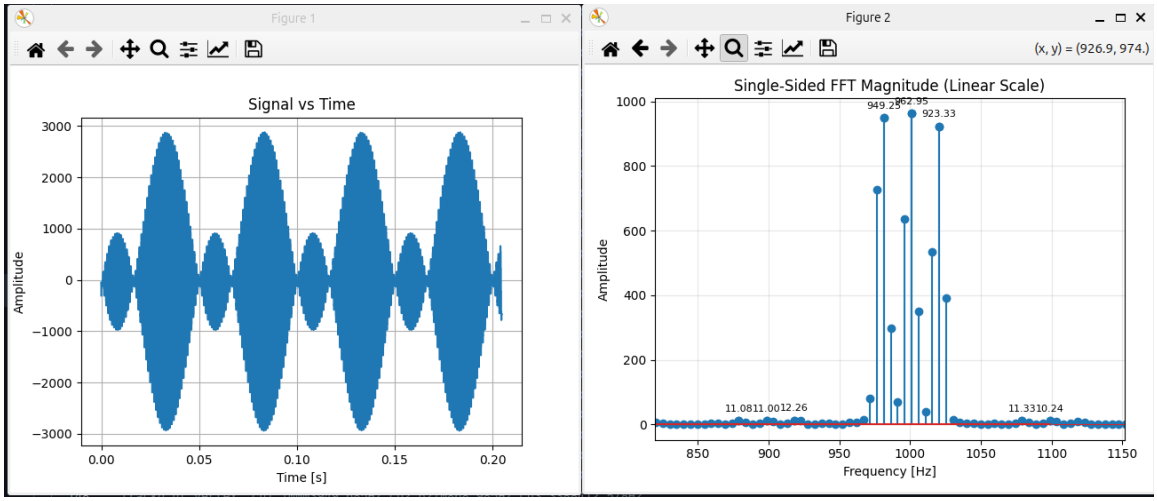


Figure 13: Low-speed debug capture for tracking channel 3.

All three resonant modes, corresponding to 9 tones in total, can also be observed with the high-speed DDR3 debug capture.

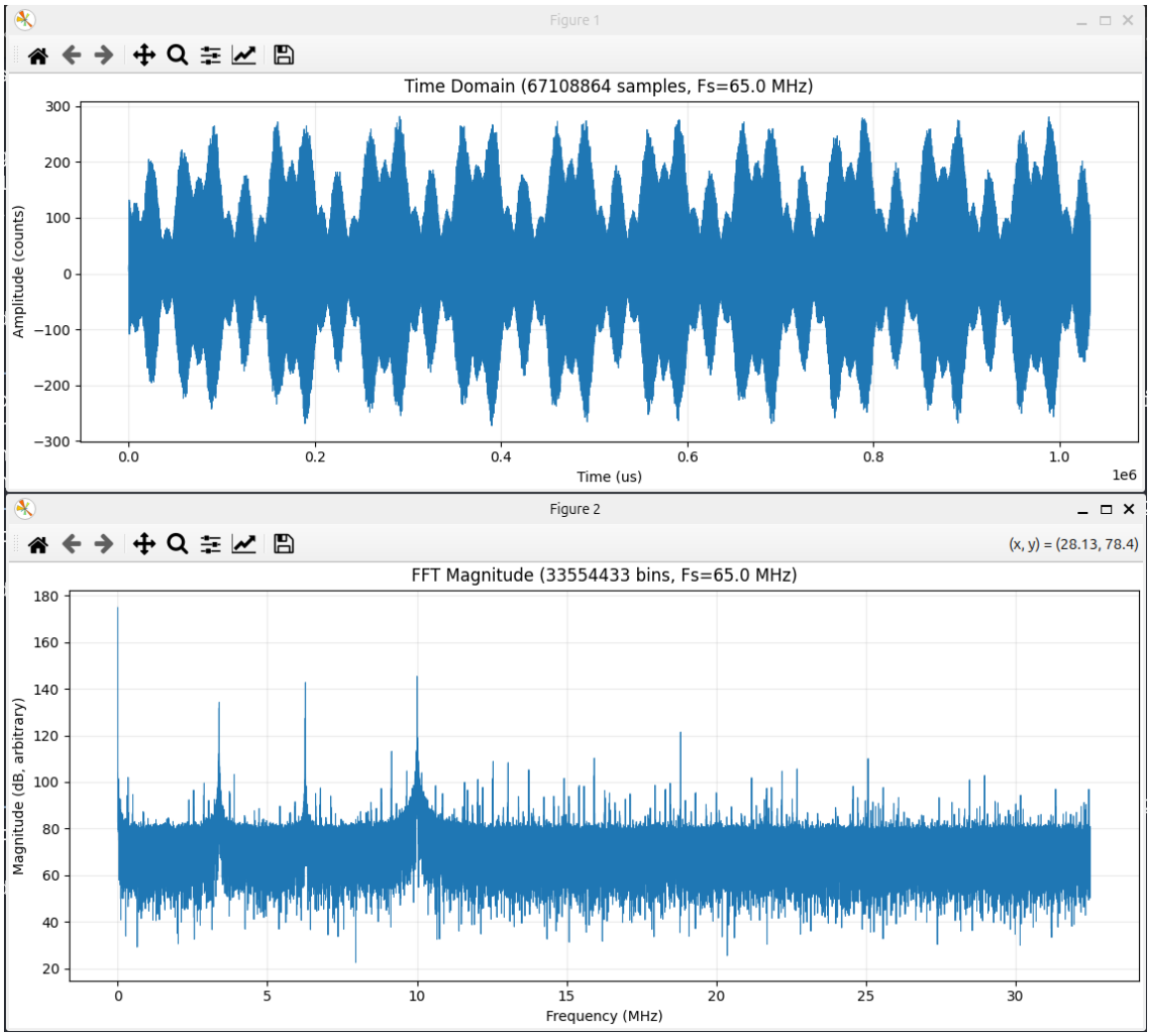


Figure 14: High-speed DDR3 debug capture showing all three tracked modes.

### 3.1.9 What to watch while testing

The tracker is only as good as the low-rate spectrum it sees. Before trusting `trackq`, check these points:

- `fft_fs` must match the actual downsampled sample rate.
- `N` must be a power of two and no larger than 2048.
- The center and side tones must be below Nyquist.
- The side spacing should be large enough to measure a slope, but not so large that the side tones stop representing the same resonance peak.
- A coarse `track3` lock should show a center tone above both side tones, with both side tones still visible.

The code is simple on purpose. It is not a full PLL and it is not trying to model the crystal. It is a measurement loop: look at three nearby points, decide which way the peak moved, and make a small CSR update.

# 4 Contribution

The ideas, architectural concepts, and signal-processing techniques presented in this report were strongly influenced by the work of Dan Gisselquist (ZipCPU). The implementation described herein was developed under his mentorship, with guidance provided through discussions of FPGA architecture, digital signal processing, and system design.

# References

[1] J. Valls, T. Sansaloni, A. Pérez-Pascual, V. Torres, and V. Almenar, "The use of cordic in software defined radios: A tutorial," *IEEE Communications Magazine*, vol. 44, pp. 46–50, Sept. 2006.